Cheat Sheet C++ → C#

Référence rapide pour transposer tes réflexes C++ vers C#. Les concepts POO/SOLID restent identiques — seule la syntaxe change.

---

1. Structure d'un fichier

C++ (déclaration dans `.h`, implémentation dans `.cpp`)

```
// MainCharacter.h
#ifndef MAIN_CHARACTER_H
#define MAIN_CHARACTER_H

class MainCharacter {
public:
    MainCharacter(std::string name, int hp);
    int getHP() const;
private:
    std::string name;
    int hp;
};
#endif
```

```
// MainCharacter.cpp
#include "MainCharacter.h"

MainCharacter::MainCharacter(std::string name, int hp)
    : name(name), hp(hp) {}

int MainCharacter::getHP() const {
    return hp;
}
```

C# (tout dans un seul fichier `.cs`, pas de header)

```
// MainCharacter.cs
namespace Roguelike.Characters
{
    public class MainCharacter
    {
        private string name;
        private int hp;

        public MainCharacter(string name, int hp)
        {
            this.name = name;
            this.hp = hp;
        }

        public int GetHP()
        {
            return hp;
        }
    }
}
```

À retenir : pas de `#include` de fichiers locaux. `using` sert à importer un `namespace`, pas un fichier précis.

---

2. Namespaces (remplace tes dossiers/organisation)

C++

```
namespace Combat {
    class CombatManager { /* ... */ };
}
// Usage : Combat::CombatManager manager;
```

C#

```
namespace Roguelike.Combat
{
    public class CombatManager { /* ... */ }
}
// Usage, depuis un autre fichier :
using Roguelike.Combat;
CombatManager manager = new CombatManager();
```

Convention C# : le namespace suit généralement l'arborescence des dossiers (`Roguelike.Characters` , `Roguelike.Skills` , etc.), exactement ce qu'on avait prévu.

---

3. Interfaces

C++ (simulée avec une classe abstraite, méthodes virtuelles pures)

```
class ITargetable {
public:
    virtual int getStat(StatType stat) const = 0;
    virtual void takeDamage(int amount) = 0;
    virtual bool isAlive() const = 0;
    virtual ~ITargetable() = default;
};
```

C# (mot-clé natif `interface`)

```
public interface ITargetable
{
    int GetStat(StatType stat);
    void TakeDamage(int amount);
    bool IsAlive();
}
```

À retenir : pas de `= 0` , pas de destructeur virtuel à gérer (le GC s'en charge), pas de `: public` — juste `interface` .

---

4. Implémenter une interface / héritage

C++

```
class MainCharacter : public ITargetable {
public:
    int getStat(StatType stat) const override { /* ... */ }
    void takeDamage(int amount) override { hp -= amount; }
    bool isAlive() const override { return hp > 0; }
};
```

C#

```
public class MainCharacter : ITargetable
{
    public int GetStat(StatType stat) { /* ... */ }
    public void TakeDamage(int amount) { hp -= amount; }
    public bool IsAlive() { return hp > 0; }
}
```

À retenir : `:` sert à la fois pour hériter d'une classe ET implémenter une interface (`class Foo : BaseClass, IInterface1, IInterface2`). Pas de mot-clé `override` obligatoire pour une interface (contrairement à l'héritage de classe abstraite, où `override` est requis).

---

5. Classes abstraites + héritage classique

C++

```
class Skill {
public:
    virtual void execute(ITargetable* caster, std::vector<ITargetable*> targets) = 0;
    virtual ~Skill() = default;
protected:
    int spCost;
};

class Ultimate : public Skill {
public:
    void execute(ITargetable* caster, std::vector<ITargetable*> targets) override {
        // logique spécifique
    }
};
```

C#

```
public abstract class Skill
{
    protected int spCost;
    public abstract void Execute(ITargetable caster, List<ITargetable> targets);
}

public class Ultimate : Skill
{
    public override void Execute(ITargetable caster, List<ITargetable> targets)
    {
        // logique spécifique
    }
}
```

À retenir : `abstract class` + `abstract` sur la méthode remplace `virtual ... = 0`. `override` est obligatoire côté classe fille.

---

6. Enums

C++

```
enum class StatType {
    ATK,
    DEF,
    HP,
    MAX_HP
};
// Usage : StatType::ATK
```

C#

```
public enum StatType
{
    ATK,
    DEF,
    HP,
    MaxHP  // convention C# : PascalCase
}
// Usage : StatType.ATK
```

À retenir : quasi identique, juste `.` au lieu de `::`, et pas besoin de `class` après `enum` (le comportement "type-safe" est natif).

---

7. Collections : `std::vector` → `List<T>`

C++

```
#include <vector>

std::vector<ITargetable*> targets;
targets.push_back(enemy);
for (ITargetable* t : targets) {
    t->takeDamage(10);
}
int count = targets.size();
```

C#

```
using System.Collections.Generic;

List<ITargetable> targets = new List<ITargetable>();
targets.Add(enemy);
foreach (ITargetable t in targets)
{
    t.TakeDamage(10);
}
int count = targets.Count;
```

À retenir : `List<T>` au lieu de `vector<T>`, `.Add()` au lieu de `.push_back()`, `.Count` (propriété, pas méthode) au lieu de `.size()`.

---

8. Pas de pointeurs — références automatiques

C++ (tu dois choisir pointeur/référence, gérer la mémoire)

```
MainCharacter* main = new MainCharacter("Hero", 100);
// ... utilisation ...
delete main;  // obligatoire, sinon fuite mémoire
```

C# (tout objet est une référence automatiquement, GC gère la mémoire)

```
MainCharacter main = new MainCharacter("Hero", 100);
// ... utilisation ...
// pas de delete, le Garbage Collector s'en charge
```

À retenir : en C#, une variable de type classe (pas `int` / `bool` /etc.) est TOUJOURS une référence en interne, même sans `*` ni `&`. Tu ne géreras jamais `delete`, `new` sert juste à construire l'objet.

---

9. Properties (remplace getters/setters manuels)

C++

```
class MainCharacter {
public:
    int getHP() const { return hp; }
    void setHP(int value) { hp = value; }
private:
    int hp;
};
```

C# (syntaxe dédiée, plus concise)

```
public class MainCharacter
{
    public int HP { get; set; }
}

// Usage :
main.HP = 100;      // équivaut à un setter
int current = main.HP;  // équivaut à un getter
```

Si tu veux de la logique dans le getter/setter (ex: empêcher HP négatif) :

```
public class MainCharacter
{
    private int hp;
    public int HP
    {
        get { return hp; }
        set { hp = value < 0 ? 0 : value; }
    }
}
```

À retenir : les `Properties` C# remplacent le pattern `getX()` / `setX()` — plus idiomatique en C# d'utiliser `main.HP` que `main.GetHP()`.

---

10. Constructeurs

C++

```
class Effect {
public:
    Effect(StatType stat, float multiplier)
        : scalingStat(stat), multiplier(multiplier) {}
private:
    StatType scalingStat;
    float multiplier;
};
```

C#

```
public class Effect
{
    private StatType scalingStat;
    private float multiplier;

    public Effect(StatType stat, float multiplier)
    {
        this.scalingStat = stat;
        this.multiplier = multiplier;
    }
}
```

À retenir : pas de liste d'initialisation (`: scalingStat(stat)`), tu assignes directement dans le corps du constructeur avec `this.`.

---

11. Modificateurs d'accès

C++	C#	Effet
`public:`	`public`	accessible partout
`private:`	`private`	accessible dans la classe seulement
`protected:`	`protected`	accessible dans la classe + classes filles
(pas d'équivalent direct)	`internal`	accessible dans le même projet/assembly seulement

À retenir : en C#, tu mets le modificateur devant CHAQUE membre individuellement (`public int hp;`), pas de bloc `public:` qui s'applique à tout ce qui suit.

---

12. Null / vérifications

C++

```
MainCharacter* main = nullptr;
if (main != nullptr) {
    main->takeDamage(10);
}
```

C#

```
MainCharacter main = null;
if (main != null)
{
    main.TakeDamage(10);
}

// Raccourci moderne C# :
main?.TakeDamage(10);  // n'exécute que si main n'est pas null
```

---

13. Exemple complet : ton `ITargetable` + `MainCharacter` en C#

Pour visualiser un morceau réel de ton architecture, transposé :

```
// Core/StatType.cs
namespace Roguelike.Core
{
    public enum StatType
    {
        ATK,
        DEF,
        HP,
        MaxHP
    }
}
```

```
// Core/ITargetable.cs
namespace Roguelike.Core
{
    public interface ITargetable
    {
        int GetStat(StatType stat);
        void TakeDamage(int amount);
        void Heal(int amount);
        bool IsAlive();
    }
}
```

```
// Characters/MainCharacter.cs
using Roguelike.Core;

namespace Roguelike.Characters
{
    public class MainCharacter : ITargetable
    {
        private string name;
        private int hp;
        private int atk;
        private int def;

        public MainCharacter(string name, int hp, int atk, int def)
        {
            this.name = name;
            this.hp = hp;
            this.atk = atk;
            this.def = def;
        }

        public int GetStat(StatType stat)
        {
            switch (stat)
            {
                case StatType.ATK: return atk;
                case StatType.DEF: return def;
                case StatType.HP: return hp;
                default: return 0;
            }
        }

        public void TakeDamage(int amount)
        {
            hp -= amount;
            if (hp < 0) hp = 0;
        }

        public void Heal(int amount)
        {
            hp += amount;
        }

        public bool IsAlive()
        {
            return hp > 0;
        }
    }
}
```

Ce squelette est du pur boilerplate structurel (pas de vraie logique de jeu dedans) — à toi de le compléter et de l'adapter avec tes propres règles.

14. Ce qui NE change PAS entre C++ et C#

- La logique POO (encapsulation, héritage, polymorphisme, interfaces) est conceptuellement identique
- Les principes SOLID s'appliquent pareil
- Le raisonnement `switch` / `if` /boucles est quasi identique syntaxiquement
- Ton architecture actuelle (`ITargetable` , `Skill` / `Effect` , `IRenderer`) reste valable telle quelle, juste traduite dans cette nouvelle syntaxe